

Презентация по лабораторной работе 7

Имитационное моделирование

Нджову Нелиа

Цель работы

Цель этой лабораторной работы - работа дискретно-событийного моделирования

1 Модель M/M/c

Задание

- 1 Модель M/M/c
- 2 Модель Росса

Модель М/М/с

Модель М/М/с(по классификации Кендалла) - это система массового обслуживания со следующими свойствами:

- М(Markovian) - входящий поток пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ .

Модель Росса

Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

Модель М/М/с

Модель М/М/с(по классификации Кендалла) - это система массового обслуживания со следующими свойствами:

- М(Markovian) - входящий поток пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ .
- М - время обслуживания каждой заявки распределено экспоненциально с параметром μ .

Модель Росса

Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

Модель М/М/с

Модель М/М/с(по классификации Кендалла) - это система массового обслуживания со следующими свойствами:

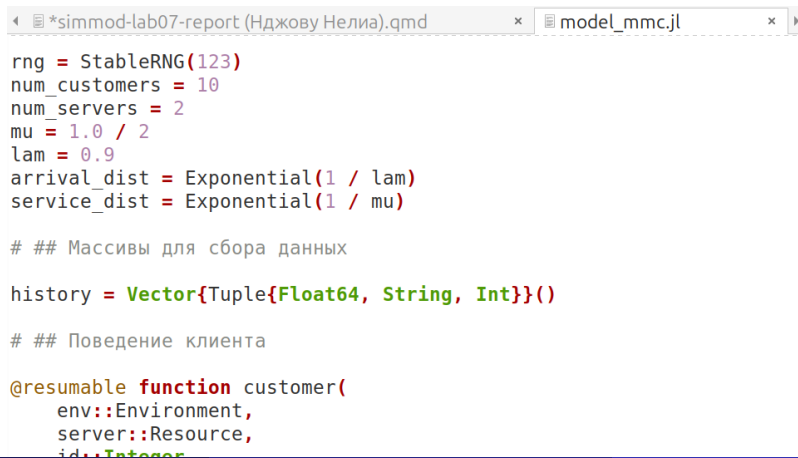
- М(Markovian) - входящий поток пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ .
- М - время обслуживания каждой заявки распределено экспоненциально с параметром μ .
- с - количество идентичных обслуживающих приборов(каналов), работающих параллельно.

Модель Росса

Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

1. Модель M/M/c

Я создала файл `scripts/model_mmc.jl` и скопировала заданный скрипт в него. Скрипт создает 10 клиентов в очереди из 2 серверов и выводит время прибытия, начала обслуживания и отправления в консоль. Я запускала код и все сработало хорошо(рис.1).



```
*simmod-lab07-report (Нджову Нелиа).qmd  model_mmc.jl

rng = StableRNG(123)
num_customers = 10
num_servers = 2
mu = 1.0 / 2
lam = 0.9
arrival_dist = Exponential(1 / lam)
service_dist = Exponential(1 / mu)

# ## Массивы для сбора данных

history = Vector{Tuple{Float64, String, Int}}()

# ## Поведение клиента

@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer
```

1. Модель M/M/c

После этого я перевожу код в структуру DrWatson(рис.2)

```
# ## Загрузка необходимых пакетов  
  
using DrWatson  
@quickactivate "project"
```

Рисунок 2: Структура DrWatson

1. Модель М/М/с

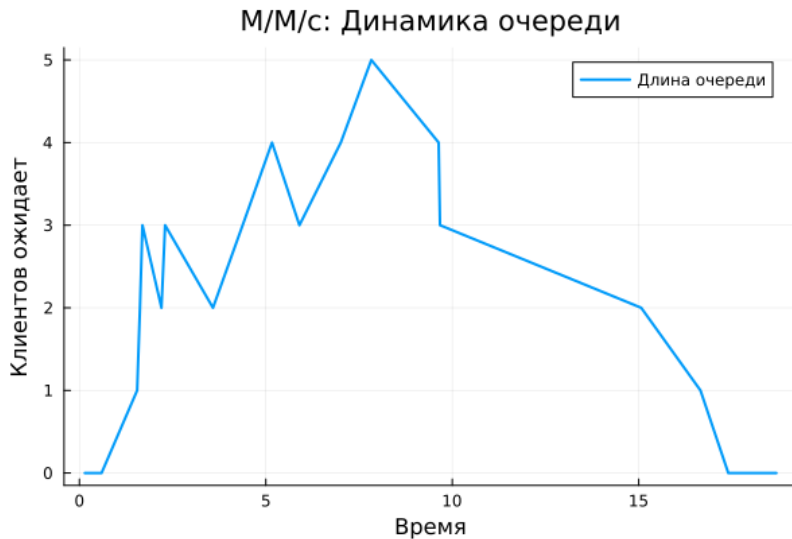
Затем я добавляю необходимую визуализацию(рис.3)

```
simmod-lab07-report (Нджову Нелиа).qmd × model_mmc.jl × model_ross.jl ×  
p1 = plot(df_results.time, df_results.n_in_system,  
          label = "Клиентов в системе",  
          xlabel = "Время",  
          ylabel = "Количество клиентов",  
          title = "М/М/с: Загрузка системы во времени",  
          linewidth = 2,  
          legend = :topright)  
display(p1)  
savefig(plotsdir("mmc_system_load.png"))  
  
# ## График 2: Длина очереди во времени  
  
p2 = plot(df_results.time, df_results.queue_length,  
          label = "Длина очереди",  
          xlabel = "Время",  
          ylabel = "Клиентов ожидает",  
          title = "М/М/с: Динамика очереди",  
          linewidth = 2,  
          legend = :topright)  
display(p2)  
savefig(plotsdir("mmc_queue_length.png"))  
  
# ## График 3: Использование серверов во времени  
  
ρ = lam / (num_servers * mu)  
p3 = plot(df_results.time, df_results.utilization,  
          label = "Использование",
```

1. Модель М/М/с

На первом графике показано количество клиентов в очереди, из графика видно, что наибольшее количество клиентов в очереди находилось в интервале 5-10 единиц времени(рис.4)

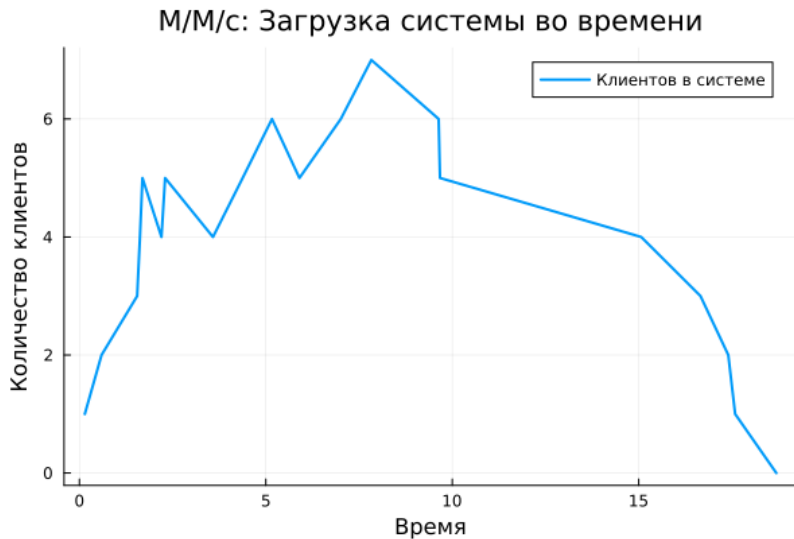
1. Модель М/М/с



1. Модель М/М/с

Второй график, который я добавил, был графиком количества клиентов в системе, из графика, аналогичного первому, мы можем видеть, что наибольшее количество клиентов находится в пределах 5-10 единиц времени(рис.5)

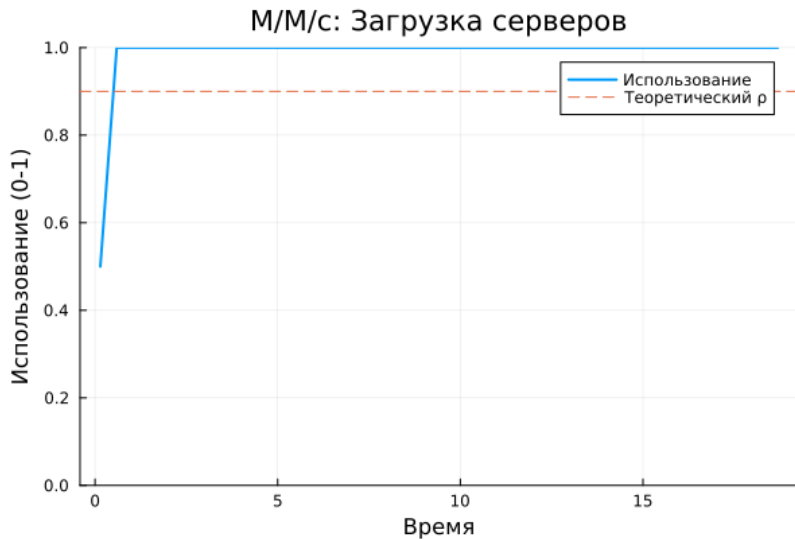
1. Модель М/М/с



1. Модель М/М/с

Третий график предназначен для проверки того, насколько загружен сервер по сравнению с теоретическим порогом (ρ), из графика мы видим, что сервер загружен больше, чем ожидалось(рис.6)

1. Модель М/М/с



2. Модель Росса

Я создала файл `scripts/model_mmc.jl` и скопировала заданный скрипт в него. Скрипт реализует систему ремонта машины с использованием запасных частей из учебника Росса. Если машина выходит из строя, она немедленно заменяется на запасную, а неисправная машина ремонтируется мастером, если запасная машина недоступна, система выходит из строя. Я запускала код и все сработало хорошо(рис.7).

simmod-lab07-report (Нджову Нелиа).qmd × model_mmc.jl × model_ross.jl ×

```
function history_to_dataframe(history::Vector{Tuple{Float64, Int, Int, Int}})
    times = [t for (t, _, _, _) in history]
    operational = [op for (_, op, _, _) in history]
    in_repair = [rep for (_, _, rep, _) in history]
    spares = [sp for (_, _, _, sp) in history]

    all_times = sort(unique(vcat(times, 0.0:1.0:maximum(times))))
    op_continuous = zeros{Int, length(all_times)}
    rep_continuous = zeros{Int, length(all_times)}
    spare_continuous = zeros{Int, length(all_times)}

    for (idx, t) in enumerate(all_times)
        last_idx = findlast(ti -> ti <= t, times)
```

2. Модель Росса

После этого я перевожу код в структуру DrWatson и добавляю несколько ремонтников(рис.8)

```
# ## Разное количество ремонтников
println("Влияние количества ремонтников")

num_repairmen_list = [1, 2, 3]
results_repairmen = []

for num_rep in num_repairmen_list
    println("\nРемонтников: $num_rep")
    crash_times = Float64[]
    for run in 1:RUNS
        crash_time, _ = sim_repair(N, S, num_rep)
        push!(crash_times, crash_time)
    end
end
```

Рисунок 8: Добавление нескольких ремонтников

2. Модель Росса

Я экспериментирую с системой, используя разное количество машин(рис.9)

```
println("\n")
println("ЭКСПЕРИМЕНТ 2: Разные конфигурации (N, S)")

machine_configs = [(10, 3), (15, 5), (20, 7)]
results_configs = []

for (N_val, S_val) in machine_configs
    println("\nКонфигурация: N=$N_val, S=$S_val")
    crash_times = Float64[]
    for run in 1:RUNS
        crash_time, _ = sim_repair(N_val, S_val, 1)
        push!(crash_times, crash_time)
    end
    avg_time = mean(crash_times)
    push!(results_configs, (N = N_val, S = S_val, avg_crash_time =
avg_time))
    println("    Среднее время краха: $(round(avg_time, digits=2))")
end
```

Рисунок 9: Разные конфигурации

2. Модель Росса

Затем я добавляю необходимую визуализацию(рис.10)

```
p1 = plot(df_dynamics_base.time, df_dynamics_base.operational,
          marker = :circle, markersize = 3, linewidth = 2,
          label = "Работающие машины",
          xlabel = "Время", ylabel = "Количество машин",
          title = "Модель Росса: N=$N, S=$S, ремонтников=1")
plot!(p1, df_dynamics_base.time, df_dynamics_base.spares,
       label = "Резервные машины", linewidth = 2, linestyle = :dash)
plot!(p1, df_dynamics_base.time, df_dynamics_base.in_repair,
       label = "Машины в ремонте", linewidth = 2, linestyle = :dot)
hline!([N], label = "Требуется работающих (N)", linestyle
       = :dashdot)
savefig(plotsdir("ross_dynamics_base.png"))

# ## Эксперимент 2: Разное количество ремонтников
println("ЭКСПЕРИМЕНТ 1: Влияние количества ремонтников")

num_repairmen_list = [1, 2, 3]
results_repairmen = []

for num_rep in num_repairmen_list
    println("\nРемонтников: $num_rep")
```

2. Модель Росса

На первом из которых показано количество работающих машин, запасных, ремонтируемых машин и необходимых ремонтников(рис.11)

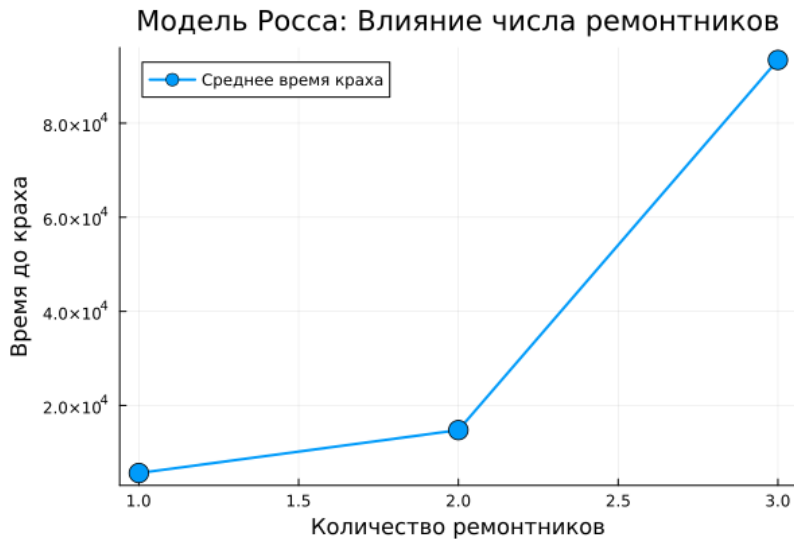
2. Модель Росса



2. Модель Росса

Из второго графика видно, что количество ремонтников влияет на среднее время простоя. Чем больше число ремонтников, тем больше времени требуется системе для выхода из строя(рис.12)

2. Модель Росса

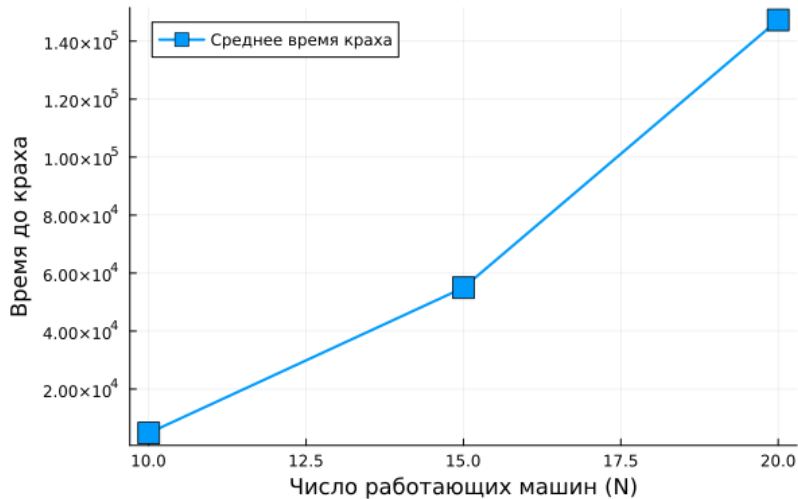


2. Модель Росса

На третьем графике показано, как количество работающих машин влияет на среднее время сбоя: чем больше количество работающих машин, тем больше времени требуется системе для выхода из строя(рис.13)

2. Модель Росса

Модель Росса: Влияние N при фиксированном S



2. Модель Росса

Когда я сравнила результаты с аналитическим методом, выяснилось, что при моделировании сбой происходит гораздо дольше, чем при аналитическом(рис.14)

```
SIMULATION vs ANALYTICAL
N=10, S=3, c=1:
  Analytical: 90.29 hours
  Simulation: 5000.0 hours
  Ratio (Sim/Anal): 55.379

N=15, S=5, c=1:
  Analytical: 82.06 hours
  Simulation: 100000.0 hours
  Ratio (Sim/Anal): 1218.564

N=20, S=7, c=1:
  Analytical: 73.19 hours
  Simulation: 60000.0 hours
  Ratio (Sim/Anal): 819.82

N=10, S=3, c=2:
  Analytical: 94.8 hours
```

В этой лабораторной работы, мы работали с дискретно-событийном моделированием